

LOOTBUGS

System Architecture, Technical Design & Subsystem Documentation

Unity 6 (6000.3.8f1)

HDRP 17.3.0

Netcode for GameObjects 2.11.2

Unity Services Multiplayer 2.2.2

Vivox Voice 16.10.0

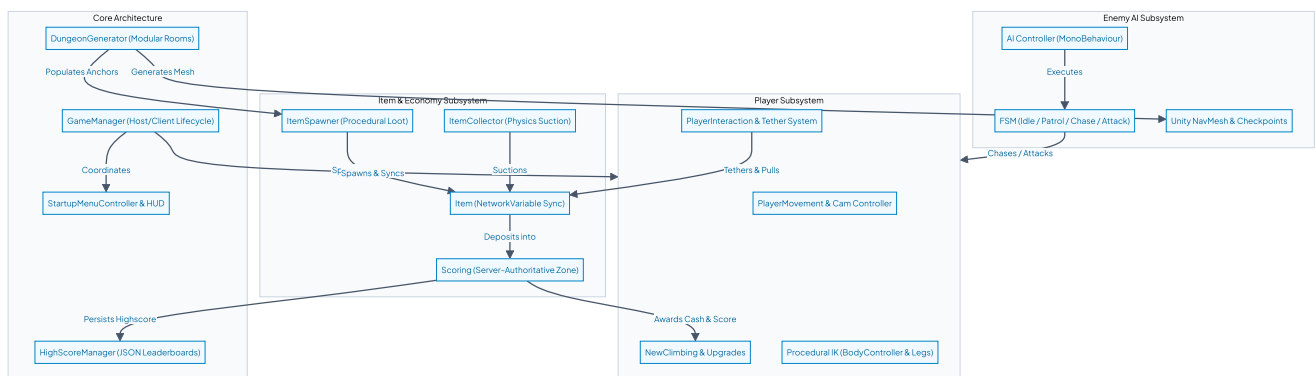
AI Navigation 2.0.12

New Input System 1.18.0

01 Executive Overview

Lootbugs is a cooperative first-person extraction and scavenging game built in Unity 6. Players navigate procedurally generated subterranean dungeons, utilize procedural insect locomotion and wall-climbing mechanics, deploy physics-based tethers and suction tools to extract rare loot, engage hostile AI entities driven by finite state machines, and bank high scores while upgrading their abilities.

FIGURE 1.1: SYSTEM ARCHITECTURE & SUBSYSTEM TOPOLOGY



02

Technology Stack & Specifications

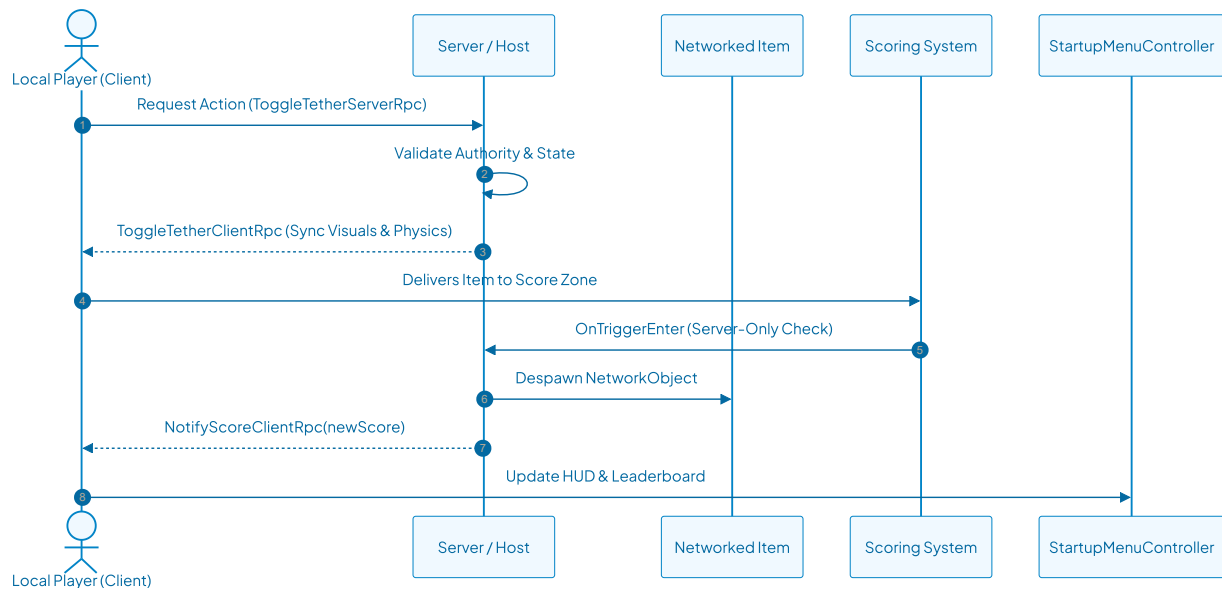
Layer	Package / Specification	Version	Architectural Role
Engine Runtime	Unity 6 Editor (C# / Mono)	6000.3.8f1	Core runtime, multi-threaded physics, and asset serialization.
Render Pipeline	High Definition Render Pipeline (HDRP)	17.3.0	High-fidelity lighting, volumetrics, custom shaders, and Volume Profiles.
Multiplayer Networking	Netcode for GameObjects (NGO)	2.11.2	Server-authoritative state synchronization, RPCs, and NetworkObjects.
Multiplayer Services	Unity Services Multiplayer & Relay	2.2.2	Cloud lobby allocation, matchmaking, and NAT punch-through relay.
Voice Communication	Vivox Voice Chat	16.10.0	In-game 3D positional audio and player voice channels.
AI & Navigation	Unity AI Navigation	2.0.12	NavMeshSurface, NavMeshAgent pathfinding, and dynamic links.
Input System	Unity New Input System	1.18.0	Action-based input abstraction and hardware device binding.
UI Framework	TextMeshPro & Unity UI	1.0.0	Canvas rendering, HUD updates, menus, and billboard world tags.

3.1 Multiplayer & Networking Layer

The multiplayer framework follows a **Server-Authoritative / Client-Predicted** paradigm using Netcode for GameObjects (NGO).

- **GameManager.cs:** Singleton **NetworkBehaviour** managing host instantiation (**StartHost**), client joining (**StartClient**), and graceful session shutdown.
- **Scoring.cs:** Server-authoritative extraction zone. Runs on the server to validate item collections, broadcasts **NotifyScoreClientRpc** to all clients, distributes currency to player climbing profiles, and despawns networked item instances.
- **NetworkTetherSystem (Tether.cs):** Synchronizes physical grapple lines between players and items using **ToggleTetherServerRpc** and **ToggleTetherClientRpc** with networked LineRenderers.

FIGURE 3.1: NETCODE CLIENT/SERVER SEQUENCE DIAGRAM



3.2 Player & Procedural Kinematics

Physics & Control:

- **PlayerMovement.cs:** Rigidbody controller handling ground damping, jump cooldowns, air multiplier, and sprint modes.
- **NewClimbing.cs:** Surface normal raycast detection allowing wall climbing, tracking stamina, and managing player economy multipliers.
- **PlayerCam.cs:** First-person mouse smoothing and field-of-view adjustments.

Procedural Bug Animation:

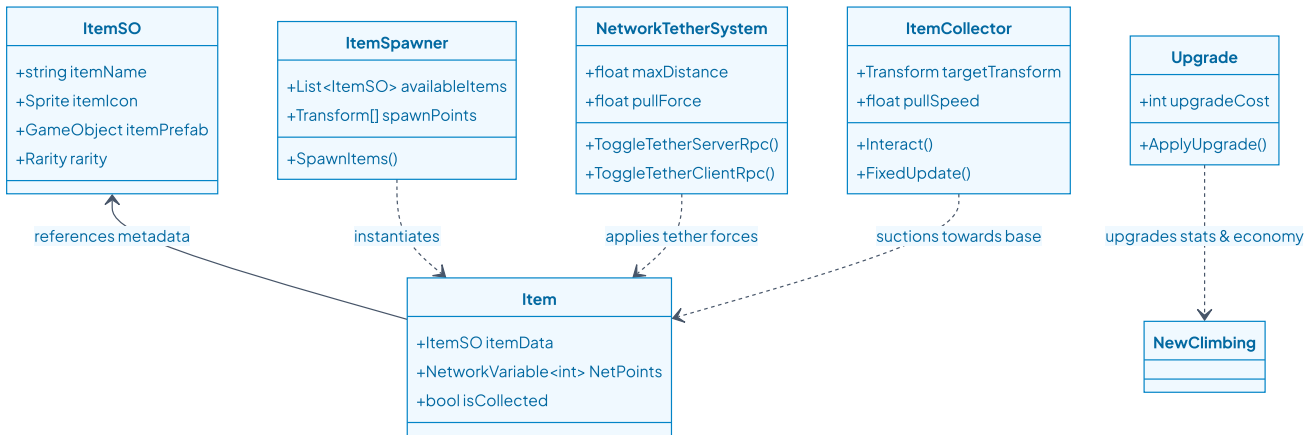
- **BodyController.cs:** Dynamic procedural solver aligning insect torso height and tilt to terrain normal vectors.
- **LegAimGrounding.cs:** Raycasts leg targets down to surfaces for authentic multi-legged crawling.

3.3 Item & Economy Subsystem

Data-Driven Loot Architecture:

- **ItemSO.cs:** ScriptableObject container specifying item names, icons, prefabs, and rarity tiers (Common to Legendary).
- **Item.cs:** Physical loot entity holding synchronized point values via `NetworkVariable<int>`.
- **ItemSpawner.cs:** Procedurally seeds loot items across designated room spawn anchors.
- **ItemCollector.cs:** Suction vacuum funnel pulling nearby physics items toward the extraction collector.
- **Upgrade.cs:** In-game store allowing players to exchange currency for speed and score perks.

FIGURE 3.2: ITEM & ECONOMY CLASS RELATIONSHIPS

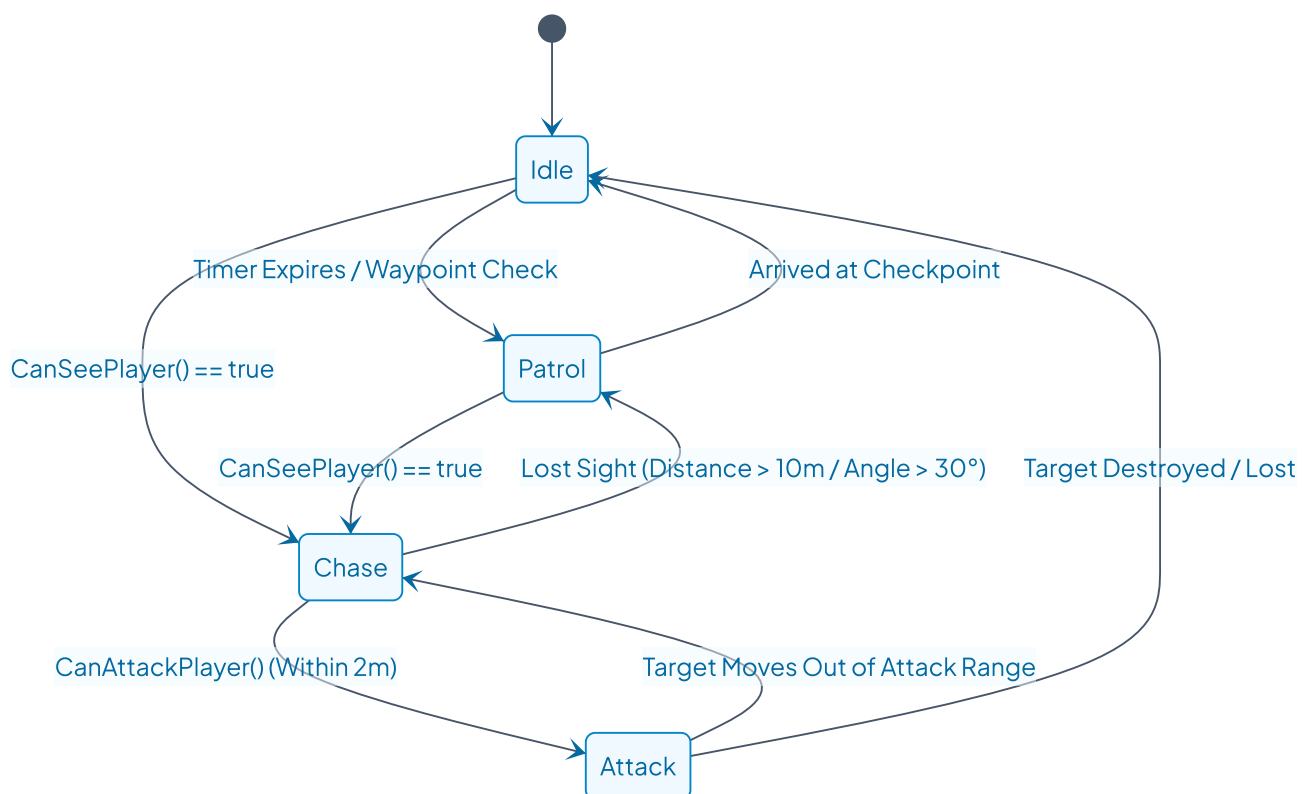


3.4 Enemy AI & Navigation Subsystem

The AI architecture uses a decoupled **Finite State Machine (FSM)** integrated with Unity AI Navigation (`NavMeshAgent`).

- **AI.cs:** Host component running the active state machine cycle in `Update()` .
- **State.cs:** Base class providing `Enter` , `Update` , `Exit` transitions, vision cone checks (`CanSeePlayer` : 10m distance, 30° cone), and melee range validation (`CanAttackPlayer` : 2m).
- **States:** `Idle` (scans for targets), `Patrol` (cycles checkpoints via `GameEnvironment`), `Chase` (high-speed pursuit), and `Attack` (melee damage).

FIGURE 3.3: ENEMY AI FINITE STATE MACHINE LIFECYCLE



3.5 Procedural Map Generation

DungeonGenerator.cs: Constructs interconnected modular layouts at runtime.

- Instantiates an initial starter room at origin and extracts door connectors tagged with `doorTag` .
- Iteratively selects open anchors and candidate room prefabs, computing inverse rotation transforms (`Quaternion.LookRotation(-targetAnchor.forward, Vector3.up)`) to snap doorways together seamlessly.

```
Lootbugs/
├── Assets/
│   ├── Blocks/                # Modular geometry blocks
│   ├── LeartesStudios/        # High-fidelity 3D environment meshes
│   ├── Materials/             # Global HDRP physical materials
│   ├── prefabs/               # Reusable game prefabs
│   │   ├── Items/            # Collectible loot items (Common → Legendary)
│   │   ├── Rooms/            # Modular room prefabs with door connectors
│   │   └── Player.prefab      # Networked player prefab with IK & cameras
│   ├── Scenes/                # SampleScene.unity (Main game world)
│   ├── Scripts/               # C# Source Code
│   │   ├── Enemy/            # AI, State Machine, and Patrol Nodes
│   │   │   ├── states/        # State implementations (Idle, Patrol, Chase, Attack)
│   │   │   ├── AI.cs          # Core AI driver component
│   │   │   └── GameEnvironment.cs # Navigation checkpoint registry
│   │   ├── Inventory/         # Loot, Items, Tether, and Upgrades
│   │   │   ├── Item.cs        # Networked item instance
│   │   │   ├── ItemSO.cs      # Item ScriptableObject definition
│   │   │   ├── ItemSpawner.cs # Room item placement controller
│   │   │   ├── Tether.cs       # Networked tethering physics
│   │   │   └── Upgrade.cs      # Player upgrade shop controller
│   │   ├── MapGeneration/     # Modular dungeon generator
│   │   ├── Multiplayer/        # GameManager & Canvas billboards
│   │   ├── PlayerMovement/     # Rigidbody movement, climbing, and procedural IK
│   │   │   └── Procedural Movement/ # BodyController.cs & LegAimGrounding.cs
│   │   ├── UI/                # StartupMenuController.cs & HighScoreManager.cs
│   │   ├── ItemCollector.cs    # Vacuum suction zone
│   │   └── Scoring.cs          # Server-authoritative scoring
│   └── Settings/              # HDRP Graphics & Volume profiles
├── AGENTS.md                  # Master AI Agent Guidelines
├── ARCHITECTURE.md             # Comprehensive Architecture Guide
└── UNITY.md                   # Dedicated Unity Reference Rules
```

Key Architectural Principles

- 1. Server Authority:** Validates scoring and item despawning on host.
- 2. Decoupled FSM:** AI states are modular classes, not monolithic switch statements.
- 3. Data-Driven:** Items are defined in ScriptableObjects ([ItemSO](#)).
- 4. Procedural IK:** Dynamic ground alignment without baked animation trees.
- 5. Zero-Alloc Loops:** No LINQ, boxing, or heap allocations in [Update](#) / [FixedUpdate](#) .